Measurement-based uncomputation of quantum circuits for modular arithmetic

Alessandro Luongo*†, Antonio Michele Miti‡, Varun Narasimhachar§, Adithya Sireesh¶
*Center for Quantum Technologies (CQT), Singapore
†Inveriant Pte. Ltd., Singapore
‡Department of Mathematics, Sapienza Università di Roma, Rome, Italy
§Agency for Science, Technology and Research (A*STAR), Singapore
¶Quantum Software Lab, University of Edinburgh, United Kingdom

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132981

***Abstract*—Measurement-based uncomputation (MBU) is a technique used to perform probabilistic uncomputation of quantum circuits. We formalize this technique for the case of single-qubit registers, and we show applications to modular arithmetic. Using MBU, we reduce Toffoli count and depth by 10% to 15% for modular adders based on the architecture of [1], and by almost 25% for modular adders based on the architecture of [2]. Our results have the potential to improve other circuits for modular arithmetic, such as modular multiplication and modular exponentiation, and can find applications in quantum cryptanalysis.**

***Index Terms*—Modular arithmetic, Quantum computing, Circuit optimization, Qubit, Design Automation.**

I. Introduction

Given an invertible Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, a quantum circuit implementation is given by a unitary gate U_f mapping $|x\rangle_n \mapsto |f(x)\rangle_m$. In general, computing f may require an out-of-place implementation, where U_f maps $|x\rangle_n |0\rangle_m$ to $|x\rangle_n |f(x)\rangle_m$, possibly introducing auxiliary qubits in a garbage state, $|g(x)\rangle_k$, which must be uncomputed to preserve reversibility. As it turns out, out-of-place implementations with garbage often arise in binary arithmetic. For instance, a *ripple carry* addition algorithm computes the carry bitwise from the least significant bits of the addends, leaving the carry in memory. To ensure correctness, the ancillary carry bit must be uncomputed from the registers. Other operations, such as (modular) addition, subtraction, and comparison by a constant, can be built from these adder gates. Efficient quantum arithmetic circuits are pivotal for advancing quantum computing, underpinning numerous key algorithms. Effective implementations of binary arithmetic are fundamental to many quantum algorithms, such as quantum algorithms for factoring [3], [4] and discrete logarithm [5], quantum walks [6], and for algorithms that construct oracles for Grover's search and quantum phase estimation, e.g. [7]–[9]. Therefore, finding cheaper ways of performing uncomputation on qubits storing garbage/intermediate computations is essential for efficient quantum circuit design.

a) Contributions: In this work, we enhance quantum circuits for modular arithmetic, specifically focusing on modular addition two n-bit quantum registers. Our approach has the potential to improve not only modular multiplication and modular exponentiation but also quantum circuits used in cryptographic attacks, see for example [1], [10], [11]. In Section II we formalize the measurement-based uncomputation technique introduced in [12]–[14] providing with Lemma 1 a plug-and-play result that can be applied to any single-qubit garbage. In Section III, we introduce a modular and composable framework to construct arithmetic circuits from other known arithmetic primitives. In particular, we quantify the resource cost for some previously known adder implementation circuits for addition, namely [1], [12], [15], [16]. We also discuss how to compute subtraction, comparison, and modular addition together with their controlled and constant variants. Section IV is dedicated to deriving formal statements for modular addition circuits optimized by MBU. Notably, table V provides resource counts for modular addition, and highlights the cost reductions achieved by using MBU. For the controlled modular adder derived from [15], MBU can reduce the Tof (Toffoli) count by $n + \frac{1}{2}$ gates in expectation. For modular addition by a constant, MBU reduces the Tof count by n gates in expectation (see theorem 4), leading to a 16.7% improvement on the original circuit. We obtain similar improvements for controlled modular addition by a constant. Finally, to our knowledge, we are the first to discuss a quantum circuit to compute whether a given register's value is contained within two other register's values (theorem 6). This circuit is a prime candidate for MBU, leading to a nearly 16.7% reduction in the cost of the circuit.

b) Notation and convention: As in [15], we denote the quantum registers by capital letters A, B, C, etc. Each register is encoded with several qubits; we denote by A_i the i-th qubit of register A. Where necessary, we put the number of qubits in a register as a subscript following the ket representing the register's state, e.g. $|x\rangle_n$ for an n-qubit register. For an n bit string $\mathbf{x} = x_{n-1}x_{n-2}\dots x_0 \in \{0,1\}^n$ we shall often equate it with the decimal number x encoded in binary by $\mathbf{x}$. We denote with $a|_n$ the n bit string encoding the number $a \in \{-2^n, \dots, 0, 1, \dots, 2^{n-1}-1\}$ in the 2's complement notation. For a number $a \in \mathbb{N}$, $|a|$ denotes the Hamming weight of the binary representation of a. We define the function $\mathbf{1}[\mathsf{condition}]$ to be 1 if $\mathsf{condition}$ is true, otherwise is 0. We call *bit string addition* the binary operator $+ :\ \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^{n+1}$, $(\mathbf{x},\mathbf{y}) \mapsto \mathbf{s}$ defined bit-wise by $s_i = x_i \oplus y_i \oplus c_i$, where the bit c_i, called i-th carry, is defined recursively by $c_0 = 0$ and $c_{i+1} = \text{maj}(x_i, y_i, c_i)$. The function maj is a logic gate called *majority* defined by

$$\text{maj}(x_i, y_i, c_i) = x_i y_i \oplus x_i c_i \oplus y_i c_i \ , \tag{1}$$

which gives 1 whenever at least half of the inputs (at least two out of three) entries are different 1. We call *1's complement* the operator $\overline{\cdot} :\ \{0,1\}^n \to \{0,1\}^n$, $\mathbf{x} \mapsto \overline{\mathbf{x}}$ defined bit-wise as $(\overline{\mathbf{x}})_i = x_i \oplus 1$. We call *2's complement* the operator defined as $\overline{\overline{\mathbf{x}}} = \overline{\mathbf{x}} + \mathbf{1}$, where $\mathbf{1}$ denotes the n bits string $0\dots0\,1$. We call *bit string subtraction* the binary operator $- :\ \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^{n+1}$, $(\mathbf{x},\mathbf{y}) \mapsto \overline{(\overline{\mathbf{x}} + \mathbf{y})}$. The string $\mathbf{d} = \mathbf{x} - \mathbf{y}$ is given bit-wise by $d_i = x_i \oplus y_i \oplus b_i$, where the bit b_i, called i-th borrow, is defined recursively by $b_0 = 0$ and $b_{i+1} = \text{maj}(x_i \oplus 1, y_i, b_i)$. Let Q be a quantum circuit composed of m gates Q_j from a gateset $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K\}$. Define the gate count $c_i := |\{j \in [m] \mid \mathsf{Q}_j = \mathcal{G}_i\}|$ for $i \in [K]$. We represent the gate counts of Q as $\mathsf{count}(\mathsf{Q}) = \{(\mathcal{G}_i, c_i) \mid i \in [K]\}$. According

Corresponding author: asireesh@ed.ac.uk

to [17], a Quantum fourier transform QFT_m has a circuit size of $\mathsf{count}(\mathsf{QFT}_m) = \{(\mathsf{H}, m)\} \cup \{(\mathsf{C\text{-}R}(\theta_i), m+1-i) \mid i \in [1, m+1]\}$.

II. Ancillary garbage uncomputation

In this section, we dive deeper into the importance of uncomputation. Consider a quantum circuit U_f computing a Boolean function $f : \{0,1\}^n \to \{0,1\}^m$ between bit strings. When f is invertible (meaning that f is injective and $n = m$) we say that U_f computes f *in-place* if it performs the mapping

$$U_f : |x\rangle_n \mapsto |f(x)\rangle_m \tag{2}$$

for any bit string $x \in \{0,1\}^n$, where we use the subscripts to denote the number of qubits in a quantum register. An in-place implementation is not always possible. In the most general case, one needs to compute f *out-of-place*, meaning that the circuit U_f realises the following mapping

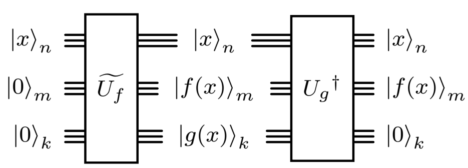
$$U_f : |x\rangle_n |0\rangle_m \mapsto |x\rangle_n |f(x)\rangle_m \ . \tag{3}$$

However, a more realistic implementation of the above gate will, in principle, involve a number k of auxiliary qubits which get mapped in a *garbage* state $|g(x)\rangle$ as follows:

$$\widetilde{U}_f : |x\rangle_n |0\rangle_m |0\rangle_k \mapsto |x\rangle_n |f(x)\rangle_m |g(x)\rangle_k \,. \tag{4}$$

The auxiliary qubits are entangled with the other registers and must be uncomputed, i.e. restored to their original value, in order to get a reversible out-of-place implementation of f. Such uncomputation can be achieved in several ways. A first solution, due to [18], simply prescribes copying the output $|f(x)\rangle_m$ into an auxiliary m-qubit register via a set of m CNOT gates and then applying $\widetilde{U}_f^\dagger$. This strategy doubles the resource cost of the reversible computation of f, also requiring m more ancillary qubits. Alternatively, one could look for an out-of-place implementation of the garbage function U_g, seeing g as a function of x and $f(x)$. When this unitary is available, one can implement U_f as in figure 1.

Fig. 1: Garbage gate uncomputation.



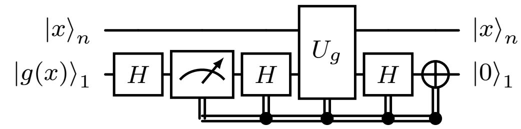
In principle, invoking the garbage circuit U_g can be quite resource intensive. Therefore, it is necessary to develop techniques to optimize the synthesis of quantum circuits and their uncomputation.

A. Measurement-Based Uncomputation (MBU).

Measurement-based uncomputation (MBU) was developed to efficiently address the challenge of uncomputing Boolean functions. This technique performs uncomputation probabilistically, by measuring the qubits slated for uncomputation on the X basis. Depending on the measurement outcome, the uncomputation circuit may either be executed or considered already completed, which helps to minimize overall quantum resource requirements. MBU has already been applied in various contexts to improve resource efficiency in uncomputation [12]–[14], [19]. However, to our knowledge, ours is the first formalization of this technique, enabling a clearer understanding and easy application in diverse contexts, as we show in Section III.

Lemma 1 (Measurement-based uncomputation lemma)**.** *Let $g : \{0,1\}^n \to \{0,1\}$ be a function, and let U_g be a self-adjoint $(n+1)$-qubit unitary that maps $U_g |x\rangle_n |b\rangle_1 = |x\rangle_n |b \oplus g(x)\rangle_1$.*

Fig. 2: A circuit for measurement-based uncomputing the unitary U_g.



There is an algorithm performing the mapping $\sum_x \alpha_x |x\rangle_n |g(x)\rangle_1 \mapsto \sum_x \alpha_x |x\rangle_n |0\rangle_1$ using U_g, 2 H gates, and 1 NOT gate with $1/2$ probability. It also involves one single-qubit computational basis measurement and a single H gate.

Proof. The algorithm is described in figure 2. We first apply the Hadamard gate on the register G and measure it in the computational basis. Since $g(x) \in \{0,1\}$, the resulting outcome is 0 or 1 with equal probabilities. If we obtain 0, we successfully uncomputed the register, leaving the registers in the state $\sum_x \alpha_x |x\rangle_A |0\rangle_G$. On the other hand, if we get the outcome 1, the state at this point is $\sum_x \alpha_x (-1)^{g(x)} |x\rangle_A |1\rangle_G$. We then apply another Hadamard to G, followed by U_g. This performs a phase kickback, leaving the quantum computer in the state

$$\begin{aligned}\sum_x \alpha_x (-1)^{g(x)} |x\rangle_A \frac{|0 \oplus g(x)\rangle_G - |1 \oplus g(x)\rangle_G}{\sqrt{2}} \\ = \sum_x \alpha_x (-1)^{g(x) \oplus g(x)} |x\rangle_A |-\rangle_G \ .\end{aligned}$$

Applying on G another Hadamard and a NOT gate results in the final state $\sum_x \alpha_x |x\rangle_A |0\rangle_G$. □

Applying the MBU lemma leads to gate costs expressed *"in expectation"*. The expected value is calculated based on a Bernoulli distribution with a probability $p = \frac{1}{2}$. This is because, for any quantum state $\sum_x \alpha_x |x\rangle_n |g(x)\rangle_1$, if $g(x) \in \{0,1\}$, when the last register is measured in the X basis, there is an equal probability of obtaining a $|0\rangle$ or $|1\rangle$ state.

III. Quantum Arithmetic

Early proposals for quantum addition [1], [15], [20], some of which are employed in fault-tolerant algorithms with state-of-the-art resource estimates [4], [21], [22], were inspired by classical circuits for addition, with an emphasis on minimizing the T count and/or the Tof (Toffoli) count, and the number of ancilla qubits. Subsequent approaches have explored more quantum-specific methods of performing arithmetic [2], [16], [23], [24]. All these arithmetic circuits serve as building blocks for more complex operations such as modular addition, multiplication, and exponentiation, to name a few. A comprehensive survey on quantum adders can be found in [11], [25].

We define a *(constant*[1]*, controlled) quantum adder* as any unitary gate that implements the addition of n-bit strings according to circuits in figures 3, 4, and 5. Note that the initial extra qubit, prepared in $|0\rangle$, in the second register is needed to take into account the possible overflow. The circuit can perform unsigned integer addition by interpreting the bit string x as its corresponding decimal number, as well as signed integer addition, using the 2's complement representation. Table I summarizes the gate and qubit counts for various quantum implementations of bit string addition from the literature.

[1]By constant we mean a classically known value employed in the quantum computation.

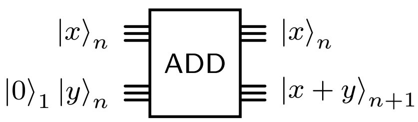


Fig. 3: (Plain) adder.

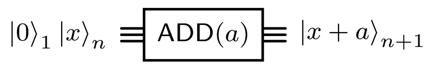


Fig. 4: Constant adder.

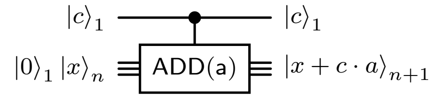


Fig. 5: Controlled constant adder.

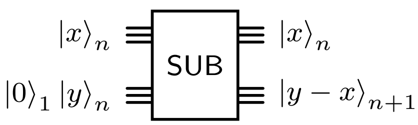


Fig. 6: (Plain) subtractor.

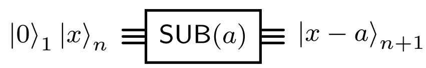


Fig. 7: Constant subtractor

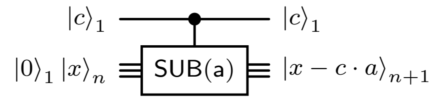


Fig. 8: Controlled constant subtractor.

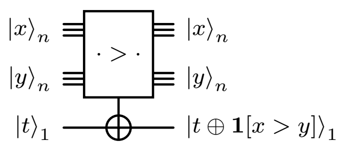


Fig. 9: Comparator.

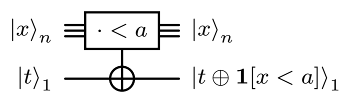


Fig. 10: Constant comparator.

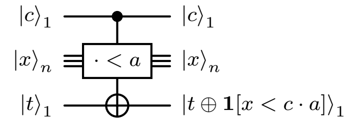


Fig. 11: Controlled constant comparator.

Procedure	**Toff. Count**	**Ancillas**	**CNOT**
VBE plain adder [1]	$4n$	n	$4n+4$
CDKPM plain adder [15]	$2n$	1	$4n+1$
Gidney plain adder [12]	n	n	$6n-1$
	QFT_{n+1}	**Ancillas**	
Draper [16]	3	0	–

TABLE I: Plain adders, gate count.

One can easily construct a controlled quantum adder circuit using an existing n-bit quantum adder. The construction requires $n+s$ ancilla qubits and $r+2n$ Toffoli gates, where s and r are the number of ancilla and Toffoli gates used by the original adder. It involves loading the value $c{\cdot}x$ in a register initially in the $|0\rangle_n$ state, performing plain addition, and unloading the control using a sequence of Toffoli gates. An improvement can be achieved by implementing Gidney's adder [12] by loading the value $c \cdot x$ via temporary logical-AND and then uncomputing it using measurements. Alternatively, the [15] adder implementation can be easily modified to perform controlled addition with a single ancilla qubit. To do so, it is sufficient to substitute a suitable CNOT in the "unmajority" gate with a Tof gate on the control register. Regarding the Draper implementation [20], circuit symmetries can be exploited to avoid controls on all gates. Only the central adder gate would need to be controlled, whereas the quantum Fourier transforms at the beginning and the end of the circuit remain uncontrolled. In total, this would require a single ancilla and n extra Tof gates. See table II for the gate count of some well-known implementations.

Procedure	**Tof Count**	**Ancillas**	**CNOT**
CDKPM	$3n$	1	$4n+1$
Gidney	$2n$	$n+1$	$7n-1$
	Tof Count	**Ancillas**	QFT_{n+1}
Draper	n	1	3

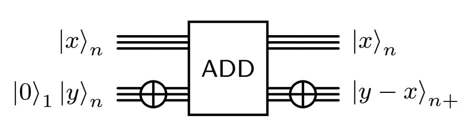
TABLE II: Controlled addition, gate count.

As suggested in [15], one can implement a quantum circuit that adds a classically known constant value a to an n-bit quantum register using any given adder circuit by loading the constant a (via $|a|$ NOT gates) on an ancillary register, performing the addition and then unloading a using the same NOT gates. In the case of Draper's QFT adder, Beauregard [2] introduced a method to add constants without requiring ancillas. Relying on the procedure sketched above, one can obtain a constant controlled adder starting with any plain adder implementation, and using additional $2|a|$ CNOT gates to conditionally Load/Unload the constant value. In [2], they propose another method based on Draper's implementation, which adds controls to rotation gates and achieves the same result without any ancilla qubits, with the circuit's complexity depending on the binary representation of the constant a. In table III we record the gate count for the above implementations.

Procedure	**Tof Count**	**Ancillas**	**CNOT**
CDKPM	$2n$	$n+1$	$4n+1+2\|a\|$
Gidney	n	$2n$	$6n-1+2\|a\|$
	QFT_{n+1}	**Ancillas**	$\mathsf{C}\text{-}\Phi_{\mathsf{ADD}}(a)$
Draper	2	0	1

TABLE III: Controlled addition by a constant 'a', gate count.

We define a *(controlled) constant quantum subtractor* as any unitary gate that implements the subtraction of the n strings according to the circuits in figures 6,7, and 8. Due to the unitarity of the quantum adder gate and the uniqueness of the adjoint, it follows that $(\mathsf{ADD})^\dagger$ implements a quantum subtractor. Alternatively, one can implement a quantum subtractor directly from the definition of bit-string subtraction as follows.



In principle, a subtractor has the same count of ancillas, Tof gates, and CNOT gates as the corresponding adder implementations. We define a *(controlled, constant) quantum comparator* as any unitary gate that implements the n string comparison according to the circuits in figures 9,10, and 11. Note that $\mathbf{1}[\mathsf{x} > \mathsf{y}]$ coincides with the most significant bit of $y - x$, i.e. it is 1 when $x > y$ and 0 otherwise. Given a quantum subtractor, one can easily obtain a string comparator reading the sign (i.e. the most significant bit) of the difference. In table IV we record the gate count for comparators with respect to the considered adder implementations. Given any circuit implementing

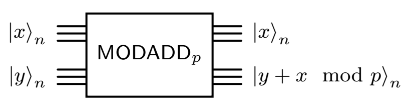
Procedure	Tof Count	Ancillas	CNOT
CDKPM	$2n$	1	$4n+1$
Gidney	n	n	$6n+1$
	QFT_{n+1}	**Ancillas**	–
Draper	6	1	–

TABLE IV: Comparator, gate count.

the comparison $(x > y)$ one can get the opposite comparison $(x \leq y)$ by simply postcomposing the first circuit with an X gate acting on the target register.

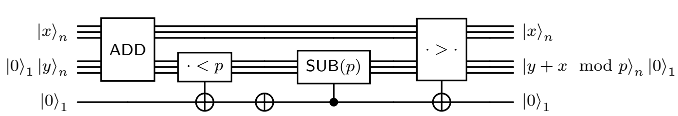
A. Quantum modular addition

Let $x \in \{0,1\}^n, y \in \{0,1\}^n$ be two bit strings, and $p \in \{0,1\}^n$ classically known with $0 \leq x, y < p$. We define a *quantum modular adder* as any unitary gate implementing the n-bit strings *addition modulo* p according to the following circuit:



A modular adder performing the operation, $|x\rangle |y\rangle \mapsto |x\rangle |x + y \pmod p\rangle$, can be decomposed into four separate subroutines [1]: quantum-quantum addition, comparison with a classical constant (by first loading in the constant using a set of X gates into an empty register, using two half subtractors [15] for the comparison, and then unloading the constant), controlled subtraction by a classical constant (by first loading the constant using a set of CNOTs into an empty register, then performing the subtraction using an inverse adder, and finally unloading the constant) and quantum-quantum comparison (using two half subtractor). The following implementation is a slight modification of [1] and [21]. It uses any available combination of adders, (controlled) subtractors (by a constant), and comparators (by a constant).

Fig. 12: Implementation of a modular adder using previous circuits.



The cost estimates for different specific combinations of the required building blocks are subsumed in table V.

IV. Measurement-based uncomputation speed-ups

In this section, we demonstrate the versatility of the MBU lemma in improving modular arithmetic circuits. While the technique is applicable to a wide range of modular arithmetic operations, we focus on a subset of key results to illustrate its impact. Specifically, we highlight their application to modular addition, controlled modular addition, modular addition by a constant, and controlled modular addition by a constant. For instance, in the modular adder architecture of VBE discussed in the previous section, the first three subroutines compute the desired value $|x + y \mod p\rangle$ in the output register, but leave an entangled ancilla qubit. The quantum-quantum comparison is required for the uncomputation of this single ancilla storing the most significant qubit of the adder circuit. Thus, the MBU Lemma can be applied to probabilistically uncompute this garbage qubit, reducing overall costs.

The theorems (thms. 1, 2, 3, 4, 5) detailed in this section represent abstract architectures of modular addition circuits which we optimize with the MBU lemma. Each subroutine in these theorems can be replaced by a specific implementation of a plain adder architecture of one's choice. In table V, we present the savings achieved by the MBU optimized modular adder, using specific plain adder architectures ([1], [12], [15], [16]) for the subroutines. Using our technique, we can derive similar results (e.g. between 15% to 25% reduction in gate count) for modular addition by a constant, controlled modular addition, and controlled modular addition by a constant.

Theorem 1 (MBU modular adder - VBE architecture)**.** *Let* $\mathsf{Q}_{\mathsf{ADD}}$ *be a quantum circuit that performs a quantum addition using* s_{ADD} *ancillas, and* r_{ADD} Tof *gates;* $\mathsf{Q}_{\mathsf{COMP}}(p)$ *be a quantum comparator using* s_{COMP} *ancillas,* r_{COMP} Tof *gates;* $\mathsf{C\text{-}Q}_{\mathsf{SUB}}(p)$ *be a quantum circuit that performs a (controlled) subtraction by a constant* p *using* $s_{\mathsf{C\text{-}SUB}}$ *ancillas, and* $r_{\mathsf{C\text{-}SUB}}$ Tof *gates;* $\mathsf{Q}'_{\mathsf{COMP}}$*: be a quantum comparator using (*s'_{COMP} *ancillas,* r'_{COMP} Tof *gates). There is a circuit,* $\mathsf{Q}_{\mathsf{MODADD}_p}$*, performing the* n*-bit modular addition operation using* $r = (r_{\mathsf{ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{COMP}}/2))$ Tof *gates in expectation and* s *ancillas with:*

$$s = 2 + \max(s_{\mathsf{ADD}}, s_{\mathsf{COMP}}, s_{\mathsf{C\text{-}SUB}}, s'_{\mathsf{COMP}})\ .$$

Proof. The first three subroutines map $|0\rangle_s |x\rangle_n |y\rangle_n$ to

$$|0\rangle_{s-1} |x\rangle_n |x + y \mod p\rangle_n |\mathbf{1}[x + y \geq p]\rangle_1\ .$$

The comparator qubit storing $\mathbf{1}[x + y \geq p]$ is finally supposed to be uncomputed using $\mathsf{Q}'_{\mathsf{COMP}}$. We can apply the MBU lemma on the comparator qubit and the uncomputation subroutine $\mathsf{Q}'_{\mathsf{COMP}}$, thus halving the cost of $\mathsf{Q}'_{\mathsf{COMP}}$ in expectation. Therefore, we finally get a cost of $r = (r_{\mathsf{ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{COMP}}/2))$ Tof gates in expectation, and the same ancilla count of the modular addition implementation. □

By suitably substituting certain adder gates with the corresponding controlled version, one obtains a routine for controlled modular arithmetic.

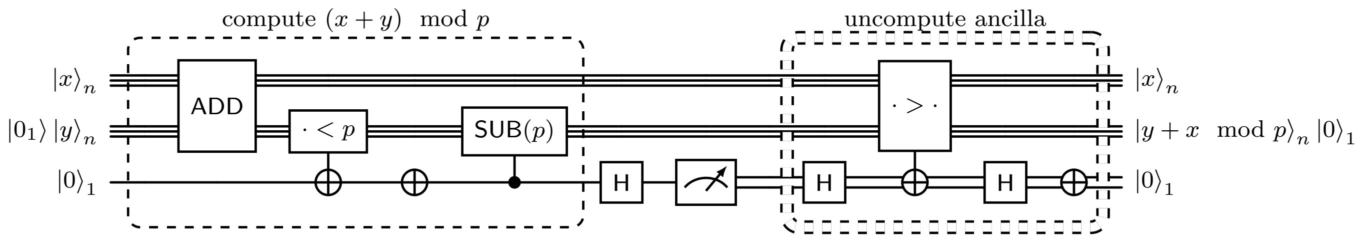
Theorem 2 (MBU controlled modular adder - VBE architecture)**.** *Let* $\mathsf{C\text{-}Q}_{\mathsf{ADD}}$ *be a quantum circuit that performs a quantum (controlled) addition using* $s_{\mathsf{C\text{-}ADD}}$ *ancillas, and* $r_{\mathsf{C\text{-}ADD}}$ Tof *gates;* $\mathsf{Q}_{\mathsf{COMP}}(p)$ *be a quantum comparator by a constant* p*, using* s_{COMP} *ancillas,* r_{COMP} Tof *gates;* $\mathsf{C\text{-}Q}_{\mathsf{SUB}}(p)$ *be a quantum circuit that performs a quantum subtraction by a constant* p*, using* $s_{\mathsf{C\text{-}SUB}}$ *ancillas, and* $r_{\mathsf{C\text{-}SUB}}$ Tof *gates;* $\mathsf{C\text{-}Q}'_{\mathsf{COMP}}$*: be a controlled quantum comparator using* $s'_{\mathsf{C\text{-}COMP}}$ *ancillas and* $r'_{\mathsf{C\text{-}COMP}}$ Tof *gates. There exists a circuit* $\mathsf{C\text{-}Q}_{\mathsf{MODADD_p}}$ *that implements an* n*-bit controlled quantum modular addition using* $r = (r_{\mathsf{C\text{-}ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{C\text{-}COMP}}/\,2))$ Tof *gates in expectation and* $s = 2 + \max(s_{\mathsf{C\text{-}ADD}}, s_{\mathsf{COMP}}, s_{\mathsf{C\text{-}SUB}}, s'_{\mathsf{C\text{-}COMP}})$ *ancillas.*

Proof. The first three subroutines in figure 13 map $|0\rangle_s |c\rangle_1 |x\rangle_n |y\rangle_n$ to

$$|0\rangle_{s-1} |c\rangle_1 |x\rangle_n |c \cdot x + y \mod p\rangle_n |\mathbf{1}[c \cdot x + y \geq p]\rangle_1\ .$$

The comparator qubit storing $\mathbf{1}[c \cdot x + y \geq p]$ is finally supposed to be uncomputed using $\mathsf{C\text{-}Q}'_{\mathsf{COMP}}$. We can apply the MBU lemma here on the comparator qubit and the uncomputation subroutine $\mathsf{C\text{-}Q}'_{\mathsf{COMP}}$, thus

Fig. 13: Implementation of a MBU modular addition. The double dotted lines represent the non-computation of the ancilla, controlled on the measurement value.



halving the cost of C-$\mathsf{Q}'_{\mathsf{COMP}}$ in expectation. Therefore, we finally get a cost of $r = (r_{\mathsf{C\text{-}ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{C\text{-}COMP}}/2))$ Tof gates in expectation, and the same ancilla count as the controlled modular addition implementation. ☐

Similarly, one can get the circuit implementation of modular addition with a constant.

Theorem 3 (MBU modular addition by a constant - VBE architecture)**.** *Let* $\mathsf{Q}_{\mathsf{ADD}}(\mathsf{a})$ *be a quantum circuit that performs a constant quantum addition using* s_{ADD} *ancillas, and* r_{ADD} Tof *gates;* $\mathsf{Q}_{\mathsf{COMP}}(p)$ *be a quantum comparator using* s_{COMP} *ancillas,* r_{COMP} Tof *gates;* C-$\mathsf{Q}_{\mathsf{SUB}}(p)$ *be a quantum circuit that performs a (controlled) subtraction by a constant* p *using* $s_{\mathsf{C\text{-}SUB}}$ *ancillas, and* $r_{\mathsf{C\text{-}SUB}}$ Tof *gates;* $\mathsf{Q}'_{\mathsf{COMP}}(a)$*: be a constant quantum comparator using (*s'_{COMP} *ancillas,* r'_{COMP} Tof *gates). There exists a circuit* $\mathsf{Q}_{\mathsf{MODADD_p}}(a)$ *that implements an* n*-bit quantum modular addition by a constant using* $r = (r_{\mathsf{ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{COMP}}/2))$ Tof *gates and* $s = 2 + \max(s_{\mathsf{ADD}}, s_{\mathsf{COMP}}, s_{\mathsf{C\text{-}SUB}}, s'_{\mathsf{COMP}})$ *ancillas.*

Proof. The first three subroutines map $|0\rangle_s\,|x\rangle_n$ to

$$|0\rangle_{s-1}\,|x\rangle_n\,|x+a \mod p\rangle_n\,|\mathbf{1}[x+a \geq p]\rangle_1 \ .$$

The comparator qubit storing $|\mathbf{1}[x+a \geq p]\rangle_1$ is finally supposed to be uncomputed using $\mathsf{Q}'_{\mathsf{COMP}}(a)$. We can apply the MBU lemma here on the comparator qubit and the uncomputation subroutine $\mathsf{Q}'_{\mathsf{COMP}}(a)$, thus halving the cost of $\mathsf{Q}'_{\mathsf{COMP}}(a)$ in expectation. Therefore, we finally get a cost of $r = (r_{\mathsf{ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{COMP}}/2))$ Tof gates in expectation, and the same ancilla count as the constant modular addition implementation. ☐

Theorem 4 (MBU modular adder by a constant - in Takahashi Architecture [26])**.** *Let* $x \in \{0,1\}^n$*. Assume to know classically* $p, a \in \{0,1\}^n$ *with* $0 \leq a, x < p$*. Also, assume we have access to the following subroutines:* $\mathsf{Q}_{\mathsf{SUB}}(\mathsf{a})$ *be a quantum circuit that performs a constant quantum subtraction using* s_{SUB} *ancillas, and* r_{SUB} Tof *gates;* C-$\mathsf{Q}_{\mathsf{ADD}}(p)$ *be a quantum circuit that performs a (controlled) addition by a constant* p *using* $s_{\mathsf{C\text{-}ADD}}$ *ancillas, and* $r_{\mathsf{C\text{-}ADD}}$ Tof *gates;* $\mathsf{Q}_{\mathsf{COMP}}(a)$ *be a classical comparator using* s_{COMP} *ancillas,* r_{COMP} Tof *gates. Then, there exists a circuit* $\mathsf{Q}_{\mathsf{MODADD_p}}(a)$ *that implements quantum modular addition by a constant using* $r = (r_{\mathsf{SUB}} + r_{\mathsf{C\text{-}ADD}} + (r_{\mathsf{COMP}}/2))$ Tof *gates in expectation and* $s = 1 + \max(s_{\mathsf{SUB}}, s_{\mathsf{C\text{-}ADD}}, s_{\mathsf{COMP}})$ *ancillas.*

Proof. As seen in the Takashi constant modular adder [26], the first two subroutines map $|0\rangle_s\,|x\rangle_n$ to

$$|0\rangle_{s-1}\,|x\rangle_n\,|x+a \mod p\rangle_n\,|\mathbf{1}[x+a < p]\rangle_1 \ .$$

The comparator qubit storing $|\mathbf{1}[x+a < p]\rangle_1$ is finally supposed to be uncomputed using $\mathsf{Q}_{\mathsf{COMP}}(a)$ and a NOT gate. We can apply the MBU lemma here on the comparator qubit and the uncomputation subroutine $\mathsf{Q}_{\mathsf{COMP}}(a)$, thus halving the cost of $\mathsf{Q}_{\mathsf{COMP}}(a)$ in expectation. Therefore, we finally get a cost of $r = (r_{\mathsf{SUB}} + r_{\mathsf{C\text{-}ADD}} + (r_{\mathsf{COMP}}/2))$ Tof gates in expectation, and the same ancilla count of the [26] constant modular adder. ☐

Theorem 5 (MBU controlled modular adder by a constant - VBE architecture)**.** *Let* C-$\mathsf{Q}_{\mathsf{ADD}}(\mathsf{a})$ *be a quantum circuit that performs a controlled addition by a constant using* $s_{\mathsf{C\text{-}ADD}}$ *ancillas, and* $r_{\mathsf{C\text{-}ADD}}$ Tof *gates;* $\mathsf{Q}_{\mathsf{COMP}}(p)$ *be a quantum comparator using* s_{COMP} *ancillas,* r_{COMP} Tof *gates;* C-$\mathsf{Q}_{\mathsf{SUB}}(p)$ *be a quantum circuit that performs a (controlled) subtraction by a constant* p *using* $s_{\mathsf{C\text{-}SUB}}$ *ancillas, and* $r_{\mathsf{C\text{-}SUB}}$ Tof *gates;* C-$\mathsf{Q}'_{\mathsf{COMP}}(a)$*: be a controlled constant quantum comparator using (*$s'_{\mathsf{C\text{-}COMP}}$ *ancillas,* $r'_{\mathsf{C\text{-}COMP}}$ Tof *gates). There exists a circuit* C-$\mathsf{Q}_{\mathsf{MODADD_p}}(a)$ *that implements an* n*-bit controlled quantum modular addition by a constant using* $r = (r_{\mathsf{C\text{-}ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{C\text{-}COMP}}/2))$ Tof *gates and* $s = 2 + \max(s_{\mathsf{C\text{-}ADD}}, s_{\mathsf{COMP}}, n + s_{\mathsf{C\text{-}SUB}}, s'_{\mathsf{C\text{-}COMP}})$ *ancillas.*

Proof. A controlled modular adder by a constant in VBE architecture, the first three subroutines map $|0\rangle_s\,|c\rangle_1\,|x\rangle_n$ to

$$|0\rangle_{s-1}\,|c\rangle_1\,|x\rangle_n\,|x+c\cdot a \mod p\rangle_n\,|\mathbf{1}[x+c\cdot a \geq p]\rangle_1 \ .$$

The comparator qubit storing $|\mathbf{1}[x+c\cdot a \geq p]\rangle_1$ is finally supposed to be uncomputed using C-$\mathsf{Q}'_{\mathsf{COMP}}(a)$. We can apply the MBU lemma here on the comparator qubit and the uncomputation subroutine C-$\mathsf{Q}'_{\mathsf{COMP}}(a)$, thus halving the cost of C-$\mathsf{Q}'_{\mathsf{COMP}}(a)$ in expectation. Therefore, we finally get a cost of $r = (r_{\mathsf{C\text{-}ADD}} + r_{\mathsf{COMP}} + r_{\mathsf{C\text{-}SUB}} + (r'_{\mathsf{C\text{-}COMP}}/2))$ Tof gates in expectation, and the same ancilla count of a controlled constant modular adder. ☐

Finally, we introduce a new quantum arithmetic subroutine called the two sided comparison circuit which checks if a register $|x\rangle$ has a value in the range specified by two other quantum registers $|y\rangle$ and $|z\rangle$, and also show how its cost can be further reduced with MBU. Compared to a non-MBU implementation, we save 16.5% for the Tof gate cost.

Theorem 6 (Two-sided comparator)**.** *Let* $x, y, z \in \{0,1\}^n$*, and* $t \in \{0,1\}$ *Let us assume we have access to* $\mathsf{Q}_{\mathsf{COMP}}$*, with*

$$|x\rangle_n\,|y\rangle_n\,|t\rangle_1 \xmapsto{\mathsf{Q}_{\mathsf{COMP}}} |x\rangle_n\,|y\rangle_n\,|t \oplus \mathbf{1}[x<y]\rangle_1$$

[1]We denote by $|p|$ the Hamming weight of the bit string of the number p expressed in binary expansion. PCQFT stands for "partially-classical" quantum Fourier transform, where each controlled rotation in the QFT is replaced by a classically controlled rotation. Results with MBU are considered "in expectation". Draper (Expec) represents the repeated use of the Draper addition circuit. In expectation, the cost of the first QFT and last IQFT become negligible compared to the rest of the circuit.

	Logical Qubits	Toffoli		CNOT,CZ		X	
		w/o MBU	with MBU	w/o MBU	with MBU	w/o MBU	with MBU
(5 adder) VBE [1]	$4n+2$	$20n+10$	$16n+8$	$20n+2\|p\|+22$	$16n+2\|p\|+18$	$\|p\|+2$	$\|p\|+2.5$
(4 adder) VBE	$4n+2$	$16n+4$	$14n+4$	$20n+2\|p\|+18$	$17n+2\|p\|+15.5$	$2\|p\|+1$	$2\|p\|+1.5$
CDKPM [15]	$3n+2$	$8n$	$7n$	$16n+2\|p\|+4$	$14n+2\|p\|+3.5$	$2\|p\|+1$	$2\|p\|+1.5$
Gidney [12]	$4n+2$	$4n$	$3.5n$	$26n+2\|p\|+4$	$22.75n+2\|p\|+3.5$	$2\|p\|+1$	$2\|p\|+1.5$
CDKPM+Gidney	$3n+2$	$6n$	$5.5n$	$21n+2\|p\|+4$	$17.75n+2\|p\|+3.5$	$2\|p\|+1$	$2\|p\|+1.5$
	Logical Qubits	QFT_{n+1}		PCQFT_{n+1}		**-**	
		w/o MBU	with MBU	w/o MBU	with MBU	-	-
Draper [16]	$2n+2$	10	8	1	1		
Draper (Expect)	$2n+2$	8	6	1	1		

TABLE V: Cost of modular addition using different plain adder circuits in the VBE architecture of modular addition.

using s_{COMP} ancillas and r_{COMP} Tof gates. Assuming we also have access to $\mathsf{C\text{-}Q}'_{\mathsf{COMP}}$, with

$$|c\rangle_1 |x\rangle_n |y\rangle_n |t\rangle_1 \xmapsto{\mathsf{C\text{-}Q}'_{\mathsf{COMP}}} |c\rangle_1 |x\rangle_n |y\rangle_n |t \oplus c \cdot \mathbf{1}[x<y]\rangle_1$$

using $s'_{\mathsf{C\text{-}COMP}}$ ancillas and $r'_{\mathsf{C\text{-}COMP}}$ Tof gates. Then, there exists a circuit that performs the operation $\mathsf{Q}_{\mathsf{IN\_RANGE}}$

$$|x\rangle_n |y\rangle_n |z\rangle_n |t\rangle_1 \xmapsto{\mathsf{Q}_{\mathsf{IN\_RANGE}}} |x\rangle_n |y\rangle_n |z\rangle_n |t \oplus \mathbf{1}[x \in (y,z)]\rangle_1$$

using $s = 1 + s_{\mathsf{COMP}}$ ancillas and $r = 1.5 r_{\mathsf{COMP}} + r'_{\mathsf{C\text{-}COMP}}$ Tof gates.

Proof. We begin by first checking if $y < x$. Applying $\mathsf{Q}_{\mathsf{COMP}}$ to the state $|0\rangle_s |x\rangle_n |y\rangle_n |z\rangle_n |t\rangle_1$ one gets

$$|0\rangle_{s-1} |x\rangle_n |y\rangle_n |z\rangle_n |\mathbf{1}[y<x]\rangle |t\rangle_1 \ .$$

Next we compute whether $x < z$ controlled on $\mathbf{1}[y<x]$, and store the output in our target. We know that $\mathbf{1}[x \in (y,z)] \equiv (\mathbf{1}[y<x] \cdot \mathbf{1}[x<z])$, and since we have both the comparator values stored in memory mapping the previous state to

$$|0\rangle_{s-1} |x\rangle_n |y\rangle_n |z\rangle_n |\mathbf{1}[y<x]\rangle_1 |t \oplus \mathbf{1}[x \in (y,z)]\rangle_1 \ .$$

We finally uncompute the intermediate value $\mathbf{1}[y<x]$ using $\mathsf{Q}_{\mathsf{COMP}}$ obtaining the state

$$|0\rangle_s |x\rangle_n |y\rangle_n |z\rangle_n |t \oplus (\mathbf{1}[y<x] \cdot \mathbf{1}[x<z])\rangle_1 \ .$$

Therefore, we obtain a cost of $r = 2r_{\mathsf{COMP}} + r'_{\mathsf{C\text{-}COMP}}$. This cost can be decreased by applying the MBU lemma on the $\mathsf{Q}_{\mathsf{COMP}}$ uncomputing $\mathbf{1}[y<x]$, thus leading to a cost of $r = 1.5 r_{\mathsf{COMP}} + r'_{\mathsf{C\text{-}COMP}}$. ☐

V. Conclusions

Through this work, we aim to emphasize the importance of enhancing existing subroutines in known quantum algorithms, as these optimizations play a crucial role in refining resource estimates for algorithms with substantial practical applications. In modular arithmetic circuit, the efficiency of the circuit highly depends on the specific plain adder used in the implementation (e.g. [1], [12], [15], [16]). This can be seen by comparing the gate counts of various modular addition implementations in the columns "w/o MBU" in table V. The flexibility of our formalization for (controlled) quantum modular addition (by a constant) allows us to select any plain adder circuit for each of its four subroutines. An interesting variant we discovered optimizes the use of available ancilla qubits by applying the Gidney adder solely for the initial addition and final comparison steps, while using the CDKPM adder for constant subtraction and constant comparison. This approach results in a lower Tof count than a modular adder fully based on CDKPM adders ($6n$ vs. $8n$) and reduces the logical qubit count compared to a fully Gidney-based modular adder ($3n+2$ vs. $4n+2$). In section. II-A, we introduced the MBU lemma. We then applied MBU to different modular addition architectures to show the savings in Tof gate counts (see thms. 1, 2, 3, 4, 5). As seen in table. V, when comparing the Tof gate costs of the original implementations and their MBU optimized versions, we saw savings anywhere from $15-25\%$ in the gate costs. We stress that our techniques can similarly improve modular addition by a constant, controlled modular addition, and controlled modular addition by a constant.

This work opens up numerous promising research directions. First, the MBU Lemma can be studied and applied in the multi-qubit setting. Our improved circuit for modular addition can be plugged in the vast literature of quantum modular arithmetic subroutines, leading to savings in the gate count, see e.g. [27]–[30]. However, approximate modular addition techniques such as the coset representation [14] seem to currently be the dominant approach in scenarios that involve a large number of repeated additions, hence making a fruitful application of MBU in this setting non-trivial. Last but not least, we leave for future research the exploration of the resource implications of our new modular addition circuits when employed as subroutines in quantum algorithms.

Acknowledgements

This work started when AS was working at Inveriant Pte. Ltd. AS is supported by Innovate UK under grant *10004359*. AMM has received funding from the European Union's *Horizon 2020 research and innovation programme* under the grant agreement *N°101034324* and has been partially supported by the *Italian Group for Algebraic and Geometric Structures and their Application* (GNSAGA–INdAM). This work is supported by the National Research Foundation, Singapore, and A*STAR under its CQT Bridging Grant. We also acknowledge funding from the Quantum Engineering Programme (QEP 2.0) under grants *NRF2021-QEP2-02-P05* and *NRF2021-QEP2-02-P01*. We thank Anupam Chattopadhyay and Zeyu Fan for useful discussions. AMM and AL thank Niels Benedikter for the hospitality at Università degli Studi di Milano.

References

[1] V. Vedral, A. Barenco, and A. Ekert, "Quantum networks for elementary arithmetic operations," *Physical Review A*, vol. 54, no. 1, p. 147, 1996.

[2] S. Beauregard, "Circuit for shor's algorithm using 2n+ 3 qubits," *arXiv preprint quant-ph/0205095*, 2002.

[3] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134. [Online]. Available: https://ieeexplore.ieee.org/document/365700

[4] C. Gidney and M. Ekerå, "How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[5] M. Ekerå, "Quantum algorithms for computing general discrete logarithms and orders with tradeoffs," *IACR Cryptol. ePrint Arch.*, p. 797, 2018. [Online]. Available: https://eprint.iacr.org/2018/797

[6] A. M. Childs, “Quantum information processing in continuous time,” *Quantum Information Processing*, vol. 1, pp. 35–43, 2002. [Online]. Available: https://link.springer.com/article/10.1023/A:1019636608477
[7] D. W. Berry, C. Gidney, M. Motta, J. R. McClean, and R. Babbush, “Qubitization of arbitrary basis quantum chemistry leveraging sparsity and low rank factorization,” *Quantum*, vol. 3, p. 208, 2019.
[8] J. F. Doriguello, A. Luongo, J. Bao, P. Rebentrost, and M. Santha, “Quantum algorithm for stochastic optimal stopping problems with applications in finance,” in *17th Conference on the Theory of Quantum Computation, Communication and Cryptography*, 2022.
[9] D. Aggarwal, G. K. Brennen, T. Lee, M. Santha, and M. Tomamichel, “Quantum attacks on bitcoin, and how to protect against them,” *arXiv preprint arXiv:1710.10377*, 2017.
[10] S. Wang, S. Deb, A. Mondal, and A. Chattopadhyay, “Optimal toffoli-depth quantum adder,” *arXiv preprint arXiv:2405.02523*, 2024.
[11] S. Wang, X. Li, W. J. B. Lee, S. Deb, E. Lim, and A. Chattopadhyay, “A comprehensive study of quantum arithmetic circuits,” *arXiv preprint arXiv:2406.03867*, 2024.
[12] C. Gidney, “Halving the cost of quantum addition,” *Quantum*, vol. 2, p. 74, 2018.
[13] N. Kornerup, J. Sadun, and D. Soloveichik, “Tight bounds on the spooky pebble game: Recycling qubits with measurements,” *arXiv preprint arXiv:2110.08973*, 2021.
[14] C. Gidney, “Approximate encoded permutations and piecewise quantum adders,” *arXiv preprint arXiv:1905.08488*, 2019.
[15] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, “A new quantum ripple-carry addition circuit,” *arXiv preprint quant-ph/0410184*, 2004.
[16] T. G. Draper, “Addition on a quantum computer,” *arXiv preprint quant-ph/0008033*, 2000.
[17] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011.
[18] C. H. Bennett, “Logical reversibility of computation,” *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973.
[19] C. Gidney, “Windowed quantum arithmetic,” *arXiv preprint arXiv:1905.07682*, 2019.
[20] T. Draper, S. Kutin, E. Rains, and K. Svore, “A logarithmic-depth quantum carry-lookahead adder,” *Quantum Information and Computation*, vol. 6, no. 4&5, pp. 351–369, 2006.
[21] D. Litinski, “How to compute a 256-bit elliptic curve private key with only 50 million toffoli gates,” *arXiv preprint arXiv:2306.08585*, 2023.
[22] E. Gouzien, D. Ruiz, F.-M. Le Régent, J. Guillaud, and N. Sangouard, “Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126133 cat qubits,” *Physical Review Letters*, vol. 131, no. 4, 7 2023. [Online]. Available: http://dx.doi.org/10.1103/PhysRevLett.131.040602
[23] C. Zalka, “Shor’s algorithm with fewer (pure) qubits,” *arXiv preprint quant-ph/0601097*, 2006.
[24] Y. Yuan, C. Wang, B. Wang, Z.-Y. Chen, M.-H. Dou, Y.-C. Wu, and G.-P. Guo, “An improved qft-based quantum comparator and extended modular arithmetic using one ancilla qubit,” *New Journal of Physics*, vol. 25, no. 10, p. 103011, 2023.
[25] Y. Takahashi, “Quantum arithmetic circuits: a survey,” *IEICE TRANSACTIONS on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 92, no. 5, pp. 1276–1283, 2009.
[26] Y. Takahashi, S. Tani, and N. Kunihiro, “Quantum addition circuits and unbounded fan-out,” *Quantum Information & Computation*, vol. 10, no. 9, pp. 872–890, 2010.
[27] I. L. Markov and M. Saeedi, “Constant-optimized quantum circuits for modular multiplication and exponentiation,” *arXiv preprint arXiv:1202.6614*, 2012.
[28] R. Van Meter and K. M. Itoh, “Fast quantum modular exponentiation,” *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 71, no. 5, p. 052320, 2005.
[29] R. Rines and I. Chuang, “High performance quantum modular multipliers,” *arXiv preprint arXiv:1801.01081*, 2018.
[30] A. Pavlidis and D. Gizopoulos, “Fast quantum modular exponentiation architecture for shor’s factoring algorithm,” *Quantum Information and Computation*, vol. 14, no. 7&8, pp. 0649–0682, 2014.